

Drift-Sense — Localization Pipeline

Applied Materials Hackathon · Problem Statement 02 · Navigation-Error Recovery

Algorithm reference for the team · 7 August 2026

1. What this document is

A single-page-per-topic reference for the algorithm we agreed on, so all four of us are building against the same design. It names the exact method at every stage, the library call that implements it, the citation that defends it, and who owns it.

Every algorithmic choice below traces to a public source. That is not decoration — **30% of the score** is the generator and augmentation *with literature study*, and unjustified knobs lose those marks. If you change a method, change its citation too.

2. The problem in one box

```
INPUT  reference : uint8 (1000, 1000)  grayscale, 1 nm/px  -> 1 um field
       search   : uint8 (1000, 1000)  grayscale, 10 nm/px -> 10 um field
```

```
OUTPUT (x, y) : float, float  centre of the match, in SEARCH-image pixels
```

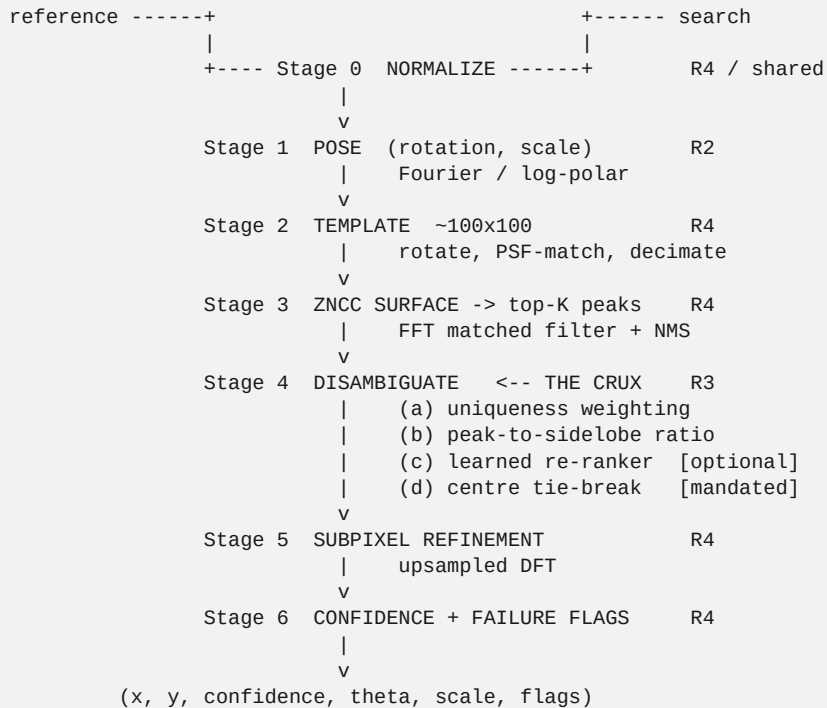
The reference occupies ~100 x 100 px inside the search image.
Search space = $(1000 - 100 + 1)^2 = 811,801$ candidate positions.

NOT fixed: scale is near 10x but not exactly 10.000x
rotation is near 0 deg but not exactly 0 deg
blur differs between the two captures

TIE-BREAK if several regions match, return the one closest to the
centre of the SEARCH image (mandated by the portal)

The fact that drives every decision: the reference and the search image are two *independent physical captures* — separate noise realizations, different PSF, possibly different detector gain and offset. Exact-pixel matching, unnormalized SSD, and hash lookup are all dead on arrival. This is why every similarity measure below is *normalized*.

3. Pipeline at a glance



4. The algorithms

#	ALGORITHM	IMPLEMENTATION	CITATION	OWNER
0	Normalize float32, per-image standardization, Difference-of-Gaussians bandpass, Hann window	<code>scipy.ndimage.gaussian_filter</code> x2; high-pass kills shading and charging gradients, mild low-pass kills noise above the useful band	Marr & Hildreth 1980	shared
1	Pose estimation Fourier–Mellin: log-polar resample of the FFT magnitude, then phase correlation	<code>skimage.transform.warp_polar</code> (scaling ='log') then <code>phase_cross_correlation</code>	Reddy & Chatterji, IEEE TIP 1996	R2
2	Template construction rotate, PSF-match, then <i>area-average</i> decimate to ~100x100	<code>cv2.warpAffine</code> , Gaussian blur, then <code>cv2.INTER_AREA</code> / <code>PIL.Image.BOX</code>	matched-filter theory	R4
3	Zero-mean normalized cross-correlation computed via FFT, normalized with integral images	<code>cv2.matchTemplate</code> (<code>TM_CCOEFF_NORMED</code>) — this is ZNCC and is already DFT-accelerated	Lewis, <i>Fast Normalized Cross-Correlation</i> , 1995	R4
3b	Peak extraction top-K local maxima with non-maximum suppression at a radius tied to the lattice pitch	<code>scipy.ndimage.maximum_filter</code> ; pitch comes free from Stage 1's spectral analysis	standard practice	R4
4a	Uniqueness weighting per-tile spectral flatness (Wiener entropy) or autocorrelation PSR → weight mask → <i>masked</i> NCC	hand-rolled masked FFT correlation — OpenCV cannot mask <code>TM_CCOEFF_NORMED</code>	Padfield, IEEE TIP 2012	R3 + R4
4b	Peak-to-sidelobe ratio PSR = (peak – mean sidelobe) / std sidelobe	direct computation over the surface outside the NMS exclusion radius; structurally identical to CFAR thresholding	radar detection theory	R3

#	ALGORITHM	IMPLEMENTATION	CITATION	OWNER
4c	Learned re-ranker (<i>optional, gated</i>) small siamese CNN scoring (template, candidate crop)	PyTorch; only K crops of ~100x100 are scored, so inference cost is negligible	—	R3
4d	Centre tie-break (<i>mandated</i>) among candidates statistically tied with the best, pick the one nearest the search-image centre	"tied" means within roughly one noise-sigma of the maximum — <i>not</i> merely all local maxima	portal requirement	R3
5	Sub-pixel refinement upsampled-DFT registration (exact sinc interpolation)	<code>skimage.registration.phase_cross_correlation(upsample_factor=100)</code>	Guizar-Sicairos, Thurman & Fienup, Opt. Lett. 2008	R4
6	Confidence calibration logistic regression on the diagnostic vector → calibrated scalar in [0, 1]	<code>sklearn.linear_model.LogisticRegression</code> fitted on our own generated data	Platt scaling, 1999	R4

5. What we deliberately are not using

Being able to say *why not* is worth as much in the presentation as the method itself.

REJECTED	WHY
SIFT / ORB / AKAZE + RANSAC	The reflex answer, and the worst possible choice here. A DRAM array produces thousands of <i>identical</i> descriptors — every via looks like every other via. Repeating texture is precisely the failure mode of descriptor matching.
SSD / unnormalized correlation	Not invariant to the gain and offset difference between two independent captures. ZNCC is invariant to any affine intensity change, which is exactly the difference we face.
End-to-end deep regression (image pair → x,y)	Runtime is inside the 50%, and the official test set is deliberately out-of-distribution and noisier than anything we generate. Heavy learned models are exactly what distribution shift punishes. Learning belongs at Stage 4c, where it re-ranks a shortlist instead of searching 811,801 positions.
LoFTR / SuperGlue	Handles periodicity well, but rejected on runtime.
Parabolic fit for sub-pixel	Fitting a parabola to three samples has a known systematic bias toward the sample centre. At a 0.5 px tolerance that bias is not academic. Zero-padding in frequency is exact interpolation of a band-limited surface, and it is citable.
Bicubic / Lanczos downsampling	Both ring at edges. Measured undershoot of 121 against a 128 background. Real detectors <i>integrate</i> over the pixel area, so area-averaging is both more accurate and more physical.

6. The idea the design turns on

Periodicity makes localization hard, but it makes pose estimation easy.

By the Fourier shift theorem, translation only changes the *phase* of the spectrum — the magnitude is unchanged. So the FFT magnitude does not care *where* the match is. Rotating the image rotates the spectrum by the same angle; scaling by s scales it by $1/s$. Under a log-polar remap both become pure translations, recoverable by phase correlation.

That means we solve rotation and scale **without knowing position**, which collapses a four-dimensional search into two cheap two-dimensional ones. And a *sharper* lattice gives *sharper* spectral peaks — so the cases that are hardest to localize are the easiest to pose. We convert the liability into an asset.

7. Engineering details that will bite

7.1 Order of operations in Stage 2 is load-bearing

Rotate at the full 1 nm/px resolution **first** — the reference is heavily oversampled there, so interpolation error is negligible. Then PSF-match. Then area-average decimate. Decimating first destroys the information the rotation needs.

7.2 Match the filters before you correlate

The two captures went through different filter chains. Downsampling the reference without matching the search image's blur injects high-frequency content that does not exist in the search image, and the correlation peak drops. Low-pass the template to the search image's effective PSF. On unseen data, estimate it from the radial power-spectrum rolloff, or scan a small set of sigma values and keep the one giving the best peak.

7.3 Pose ambiguities must be tested, not assumed away

- **180 degrees.** The magnitude spectrum of a real image is centrosymmetric, so theta and theta+180 are indistinguishable. Test both in Stage 3 and keep the better correlation.
- **90 degrees.** A square DRAM grid is symmetric under 90-degree rotation. Same resolution: test the candidates.
- **FinFET anisotropy.** A fin field gives strong spectral peaks along one axis and weak ones along the other. The estimate is still valid but noisier in the weak direction — widen the search band accordingly.
- **Fallback.** If spectral peaks are too weak, drop to a bounded grid search over (theta, scale). We are estimating a *residual* around the nominal 10x, not a scale from scratch.

Budget warning. Multi-hypothesis pose means Stages 2–3 run **2 to 4 times**, not once. At roughly 20–50 ms per ZNCC pass that consumes most of the 100 ms budget. This is what the escalation modes in section 9 exist to manage.

7.4 NMS radius comes from the lattice pitch

Suppress at a radius tied to the estimated pitch, so we retain one candidate *per lattice cell* rather than fifty clustered on a single peak. The pitch is a free by-product of Stage 1's spectral analysis.

7.5 Two sub-pixel methods, and we should measure rather than argue

(a) Upsample the ZNCC surface near the winning peak, as specified. **(b)** Take the integer peak, crop 100x100 from the search image at that location, and phase-correlate it against the template with `upsample_factor=100`. Method (b) uses Guizar-Sicairos as designed; method (a) interpolates a surface whose shape is distorted by the local-variance denominator. Both are ~10 lines. We implement (a) as primary and benchmark (b) against it — "we measured both and picked the better" is a good slide.

8. Frozen interfaces

These signatures are fixed so modules can be built and tested independently. Do not change one without telling the team.

```
# pose.py [R2]
estimate_pose(reference, search, nominal_scale=10.0) -> PoseEstimate
    """Returns theta_deg, scale, and a quality metric. Never raises."""

# matcher.py [R4]
build_template(reference, theta, scale, psf_sigma_px) -> np.ndarray
zncc_surface(template, search, weight=None) -> np.ndarray
top_k_peaks(surface, k, nms_radius) -> list[Peak]
refine_subpixel(surface, peak, upsample=100) -> tuple[float, float]

# disambiguate.py [R3]
uniqueness_map(reference, tile=64) -> np.ndarray
peak_to_sidelobe(surface, peak, exclusion_radius) -> float
select_candidate(peaks, image_centre, tolerance) -> tuple[Peak, bool]

# localize.py <-- THE SUBMITTED DELIVERABLE [R4]
localize(search: np.ndarray, reference: np.ndarray,
         mode: str = "auto") -> LocalizationResult
    """Returns x, y, confidence, diagnostics.
    MUST NEVER RAISE on valid input - degrade to a flagged
    low-confidence answer instead."""
```

Stage 6 emits these alongside (x, y):

FIELD	MEANING
ncc_peak	Absolute correlation quality
psr	Peak-to-sidelobe ratio — the ambiguity measure
n_tied	Candidates statistically tied with the best
theta_est, scale_est	Estimated pose, sanity-checkable against expectation
tie_break_used	Whether the centre rule decided the answer
uniqueness_score	Mean uniqueness of the reference — is there an anchor at all?
confidence	Calibrated scalar in [0, 1]
low_confidence_flag	Boolean for the tool to escalate

Build Stage 6 from day one, not at the end. **It is 10% of the score outright.** On a real tool a low-confidence flag would trigger a wider search or a fallback to a different alignment site — that is the production thinking a tool vendor's judges are looking for. A claimed 100% success rate scores *worse* than an honest 94% with a well-diagnosed failure mode.

9. Runtime budget and escalation

Runtime is scored inside the 50%. On a tool performing thousands of moves per day, **average** time is what matters — so adaptive compute is both correct engineering and a strong story.

MODE	WHAT RUNS	TARGET	TRIGGER
Fast	Assume scale = 10.0 and theta = 0. Single FFT-ZNCC. Stages 0, 2, 3, 5 only.	~30–60 ms	default
Robust	Full Stage 1 pose + multi-hypothesis + uniqueness weighting	~60–130 ms	Fast-mode PSR below threshold
Ambiguous	Learned re-ranker, then centre tie-break, then flag	+~20 ms	candidates statistically tied

Target: under 100 ms per pair on CPU in Robust mode. These are estimates, not measurements — we must report actual figures on named hardware. Optimization levers if needed: float32 throughout, `scipy.fft` with `workers=-1`, pad to 1024, pre-plan and reuse FFT objects across calls, `rfft2` instead of full complex FFT.

On GPUs: nothing in the source material forbids or requires one. CPU-only is *our choice* — portability, reproducibility, and credibility for a tool deployment. We must not present it as an organizer constraint.

10. Open items

QUESTION FOR THE ORGANIZERS	IMPACT IF WE GUESS WRONG
Rotation and scale ranges in the official test set	Determines the pose search band and therefore runtime
Success tolerance, and in which pixel grid	Changes the reported number by 10x
Coordinate convention: origin, axis order, indexing, pixel-centre	Systematic ~0.5 px bias if we guess wrong. Working assumption: origin top-left, x = column, y = row, 0-indexed, centre = (499.5, 499.5)
Can the reference region be clipped at a search-image edge?	Determines whether we handle partial-overlap correlation
Does the harness expect a specific function signature or CLI contract?	Packaging

All convention constants live in **one** config module so they can be switched in a single place when answers arrive. None of these block us from building.

Prepared for the Drift-Sense team. Sources: Drift-Sense Project Handbook (PS-02) and the corrected timeline and work split, rev. 2. Method choices follow Part V of the handbook; section 7.5 and the OpenCV note in section 4 are implementation recommendations to be confirmed by measurement.